

七、全游戏运行逻辑（便于画流程图）

以下按**时间顺序**描述从启动到结局的完整运行逻辑，便于据此绘制游戏运行流程图。

7.1 入口与主菜单

1. **启动**：用户执行 `main.py` / `main2.py`（CLI）或 `game_server.py` 后打开前端（Web），进入**主菜单**。
2. **主菜单分支**：
 - **新游戏** → 进入 7.2；
 - **加载存档** → 读档得到 `global_state` 与当前进度，直接进入 7.4 的「当前场景 + 选项」；
 - **存档管理**（列表/删除）→ 操作后回到主菜单；
 - **退出** → 结束。

7.2 新游戏：参数输入

3. 用户依次选择或输入：**主角属性**（如性别、年龄、性格等）→ **难度**（简单/中等/困难）→ **故事基调**（如 HE/BE/NE、黑暗、幽默等）→ **主题**（一句话或关键词）；Web 端还可选**图片风格**。
4. 参数齐备后触发**世界观生成**（7.3）。

7.3 世界观生成与展示

5. 调用 `llm_generate_global`(主题, 主角属性, 难度, 基调)（可能命中**世界观缓存**或**模板**，未命中则请求 LLM），得到 `global_state`（含 `core_worldview`、`flow_worldline`、`tone` 等）。
6. 可选：**后台异步补全完整世界观**（`template` 分阶段）。
7. 调用 `generate_ending_prediction`() 生成**隐藏的结局预测**，写入状态供后续基调修改使用。
8. **展示核心设定**给用户（主线、章节矛盾、主角/配角概要等）。
9. **主角立绘**：依赖世界观异步生成主角三视图，写入 `initial/main_character/<game_id>/`。
10. 生成**第一层剧情**：`llm_generate_local` 或等价逻辑得到「开始游戏」的**初始场景 + 2 个选项**。
11. **Web 特有**：此时可请求 **pregenerate-next-layers**，后台预生成**当前 2 选项的下一层 + 再下一层**（含可选场景图），写入 `pregeneration_cache`。
12. 进入**玩法循环**（7.4）。

7.4 玩法循环（核心）

剧情生成依赖（步骤 15「取下一段内容」及 7.3 中第一层剧情均适用）：每一步剧情生成

（`llm_generate_local` / `_generate_single_option`）**强依赖**两类输入——（1）**世界观**：`global_state` 中的 `core_worldview`（核心世界观）、`flow_worldline`（当前章节与矛盾状态）、`tone`（故事基调），要求剧情完全符合世界观与当前状态；（2）**人物形象（主角规范）**：`global_state` 中的

protagonist_canonical（主角性别、年龄、外貌等文字设定），用于约束剧情中对主角的描写与主角立绘一致。主角立绘图片本身不参与剧情 LLM 调用，但与剧情共用同一套主角规范，叙事与视觉保持一致。画流程图时可在「世界观生成」「主角立绘」到「第一层剧情」「取下一段内容」之间标出依赖箭头，体现**先有世界观与主角规范，再生成剧情**的顺序。

13. **展示当前状态**：当前场景描述、2 个选项（及可选场景图）、角色/进度面板等。
14. **用户做出选择**：选择选项 A 或 B（或 Web 端「下一段」等）。
15. **取下一段内容**：
 - **Web**：若预生成缓存中有对应 scene_id 的下一层数据，直接**从缓存取**；否则请求 **generate-option** 实时生成。
 - **CLI**：直接调用 `generate_all_options / llm_generate_local` 等得到新场景 + 新 2 选项。
16. **更新状态**：用本段输出的 `flow_update` 更新 `global_state['flow_worldline']`（如当前章节、`chapter_conflict_solved`）；若有**深层背景关联**选项被选中，则**解锁**对应角色的深层背景并记录信息差，必要时调用 `modify_ending_tone`(触发事件) 修改结局基调。
17. **配角处理**：从本段剧情中识别**出场/提及配角**，若为首次展示则触发**建档**；可选：生成配角参考图、从剧情中抽取**身份揭示**更新档案。
18. **章节检查**：若本段标记「章节矛盾：已解决」，则 `_check_chapter_conflict()`：当前章节完成 → 进入**章节深化**（对涉及角色做深层背景深化），并可将 `current_chapter` 递增、重置 `chapter_conflict_solved`。
19. **信息差检查**：若**未发现的信息差数量**达到阈值（如 5 条），调用 LLM 生成**隐藏的剧情深化内容**（揭秘/反转），自然插入后续展示。
20. **预生成 (Web)**：用户看到新场景与 2 选项后，前端可再次请求 **pregenerate-next-layers**，后台为当前 2 选项预生成下一层 + 再下一层，并执行 **cleanup_old_cache / cleanup_used_options** 清理已用缓存。
21. **结局判断**：若满足结局触发条件（如全部章节完成、用户选择结束等），则跳出循环，进入 **7.5**；否则回到步骤 13，用**新场景 + 新 2 选项**继续循环。

7.5 结局与返回

22. 调用 `generate_ending_prediction / modify_ending_content`（按当前状态）生成**结局文本**。
23. **展示结局**（HE/BE/NE 等主基调 + 具体内容）。
24. 结束后可回到**主菜单**（步骤 2）：新游戏、加载、存档管理或退出。

7.6 存档与静态资源（贯穿全程）

- **存档**：玩法循环中或主菜单可**保存**（将 `global_state`、当前场景、选项等写入 `saves/`）；**加载**即读档后回到 7.4 的对应场景。
- **静态资源**：Web 端请求主角图、场景缓存图时，由 `/initial/main_character/...`、`/image_cache/...` 等路由返回文件。

画图建议：可将「主菜单」画为起点与回流节点；「新游戏 → 参数输入 → 世界观生成 → 展示设定 → 第一层剧情」画为一条竖线；「玩法循环」画为一个**大循环框**，框内依次为：展示 → 用户选择 → 取内容（缓存/实时） → 更新状态 → 配角/章节/信息差处理 → 预生成（Web） → 结局判断（是→出框到结局，否→回到展示）；「结局展示」后箭头指回主菜单。存档/加载可画为与主菜单和玩法循环相连的侧枝。